PROJECT WEIGHIN: TECHNICAL PRD

Resource-Cost Benchmarking & CI Regression Toolkit for Soroban Smart Contracts

Author: Ijioma Makwochukwu Nissi

Date: July 2026

Status: Approved / Architecture Frozen

Target Ecosystem: Stellar / Soroban

1. Executive Summary & Problem Statement

Unlike Ethereum's single scalar gas mechanism, Stellar's **Soroban** smart contract environment uses a multi-dimensional resource metering engine tracking **11 distinct physical limits** simultaneously. Managing allocations across compute constraints, memory footprints, ledger operations, and size deltas makes cost predictability highly complex for production engineering teams.

Currently, the ecosystem lacks native developer tooling for regression detection—the equivalent of Foundry's gas snapshots in Solidity. `weighin` provides a local execution-simulation harness built in **Rust** alongside automated CI checkpoints and analytics suites engineered in **TypeScript**. This cross-language architecture delivers atomic regression gates natively within development workflows.

> **Core Objective:** Provide a complete public-good toolchain that allows developers to catch resource bloat on every pull request, preventing inefficient code deployments and stabilizing transactional rent fees before hitting mainnet pipelines.

2. The 11 Multi-Dimensional Resource Limits

Every transaction executed inside the Soroban WebAssembly (WASM) host room is metered against specific thresholds. The `weighin` core instrumentation tracks all 11 limits:

Resource Dimension	Description / Physical System Impact	Failure Point
CPU Instructions	VM processing computation cycles consumed during execution.	Out of Gas / Timeout
Memory Bytes	Peak dynamic RAM allocations requested by the sandbox context.	Allocation Fault
Ledger Read Entries	Total distinct database read operations requested from storage.	IO Cap Exceeded
Ledger Read Bytes	Cumulative payload size of data fetched from ledger storage keys.	Bandwidth Cap
Ledger Write Entries	Total keys updated or inserted into the active storage states.	State Constraint
Ledger Write Bytes	Cumulative size of raw payloads committed to storage fields.	State Constraint
Historical Read Bytes	Access requirements directed to persistent historical context records.	IO Cap Exceeded
Contract Data Hard Limit	Strict maximum threshold limits applied to instance allocations.	Instance Fault
Transaction Size Bytes	Physical weight footprint of the fully compiled transaction envelope.	Network Frame Limit
Events Count	Total operational diagnostic logs emitted during function lifecycle.	Buffer Overflow
Event Data Bytes	Aggregated dynamic weight footprint of emitted log payloads.	Buffer Overflow

3. Cross-Language Architecture & Boundaries

To optimize performance and developer utility, the system divides responsibilities between two distinct execution environments:

3.1 The Low-Level System Layer (Rust Stack)

- **weighin-core:** A standalone rust crate wrapping `soroban-env-host`. It instantiates the execution frame and queries `env.budget()` pre- and post-invocation to extract exact resource deltas.
- **weighin-cli:** The local test runner interface. It parses target WASM contract binaries, handles reproducible ledger initializations, and serializes profiling metrics to deterministic JSON targets.

3.2 The Web Automation & Analytics Layer (TypeScript Stack)

- **weighin-action:** A native Node.js-driven GitHub Action. It orchestrates baseline comparison builds across git refs and structures markdown telemetry data directly inside Pull Request workflows.
- **Dashboard Backend (Bun / Fastify):** A highly concurrent TypeScript microservice optimized for parsing, validating, and ingesting raw JSON data frames into an analytics layout.
- **Dashboard Frontend (React + Vite):** An interactive analytical client utilizing production visualization engines to render long-form multi-variable trend charts and cross-version SDK performance profiles.

4. Data Interface Contract

Because the core engine (Rust) and the automation layers (TypeScript) communicate across boundaries, data schemas are structured under a strict contract. The following JSON matrix layout is generated by the CLI and parsed via Zod validation on the backend:


```
{
  "contract_id": "CDA3...7X2",
  "git_commit": "cfb482a1768c6e269203673c914e671c6d123456",
  "soroban_sdk_version": "21.2.0",
  "timestamp": 1783178700,
  "benchmarks": [
    {
      "function_name": "process_order",
      "metrics": {
        "cpu_instructions": { "consumed": 894310, "limit": 10000000 },
        "memory_bytes": { "consumed": 1048576, "limit": 4194304 },
        "ledger_read_entries": { "consumed": 2, "limit": 40 },
        "ledger_read_bytes": { "consumed": 256, "limit": 2097152 },
        "ledger_write_entries": { "consumed": 1, "limit": 32 },
        "ledger_write_bytes": { "consumed": 64, "limit": 1048576 },
        "historical_data_read_bytes": { "consumed": 0, "limit": 4194304 },
        "contract_data_hard_limit": { "consumed": 0, "limit": 1 },
        "tx_size_bytes": { "consumed": 412, "limit": 71680 },
        "events_count": { "consumed": 1, "limit": 100 },
        "event_data_bytes": { "consumed": 32, "limit": 10240 }
      }
    }
  ]
}
```

5. Core Component Specifications

5.1 weighin-action (CI Gatekeeper)

The GitHub Action must run natively inside a Node environment on the workflow host without forcing Docker multi-stage run times.

- **Baseline Execution Flow:** On a PR event, the action checks out the target base branch, triggers a local compilation via `cargo build --target wasm32-unknown-unknown`, runs `weighin-cli` to snapshot reference metrics, swaps back to the head feature branch, repeats the analysis, and performs structural differential calculations.
- **Regression Enforcement:** If any observed delta violates conditions declared inside the local `weighin.toml` profile, the action terminates with an active exit status error code, successfully failing the PR pipeline check.

5.2 Dashboard Storage Schema (PostgreSQL Layout)

The repository layer handles indexing and structural isolation of raw metrics to power analytical time-series charts efficiently.

```
CREATE TABLE contract_snapshots (
    id BIGSERIAL PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    contract_name VARCHAR(100) NOT NULL,
    function_name VARCHAR(100) NOT NULL,
    commit_hash VARCHAR(40) NOT NULL,
    branch_name VARCHAR(100) NOT NULL,
    soroban_sdk_version VARCHAR(20) NOT NULL,

    -- Metering Blocks
    cpu_instructions BIGINT NOT NULL,
    memory_bytes BIGINT NOT NULL,
    ledger_read_entries INT NOT NULL,
    ledger_write_entries INT NOT NULL,
    ledger_read_bytes BIGINT NOT NULL,
    ledger_write_bytes BIGINT NOT NULL,
    tx_size_bytes INT NOT NULL,
    events_count INT NOT NULL,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_snapshot_lookup ON contract_snapshots (project_name, contract_name,
function_name);
CREATE INDEX idx_commit_hash ON contract_snapshots (commit_hash);
```

6. Configurable Quality Rule Framework

Projects dictate localized evaluation rules through an integrated `weighin.toml` file stored at the workspace root context.

```toml
[thresholds.global]
fail_on_any_regression = false
max_allowed_cpu_increase_pct = 5.0
max_allowed_memory_increase_pct = 2.5

[thresholds.functions.execute_swap]
ledger_write_entries = "strict_zero_tolerance"
cpu_instructions = "allow_10_percent_increase"
memory_bytes = "strict_zero_tolerance"
```

> **Security Note:** The service layer must evaluate these parameters strictly to prevent arbitrary out-of-bounds metrics from overriding pipeline execution outcomes during high-concurrency automated runner tasks.